Backend Developer — Step 5

Authentication & Security

Roadmap objective: Secure user identity, sessions/tokens, passwords and API access
Level: Beginner → Intermediate
Last reviewed: September 2026

How to use these notes

Read the concepts first, reproduce the examples yourself, complete the practical lab, and answer the interview questions without looking at the notes. Use the official documentation links as the final authority for changing APIs and platform behavior.

Learning outcomes

- Authentication vs authorization
- Password hashing
- Sessions
- JWT
- Refresh tokens
- CORS
- CSRF/XSS
- Input validation
- Secrets
- Rate limiting

Core concepts

1. Never store plaintext passwords. Store strong password hashes using a dedicated password-hashing algorithm/library.

2. Authentication answers who the user is; authorization answers what that user may do.

3. JWT is a token format, not a complete security architecture. Token storage, expiration, rotation and revocation strategy matter.

4. Validate untrusted input on the server even when the frontend already validates it.

Concept map

```
Authentication & Security
  |
  +--> Learn the concept
  |
  +--> Reproduce a small example
  |
  +--> Apply it in a feature
  |
  +--> Test failure/edge cases
  |
  +--> Document the decision
```

Detailed study guide

1. Authentication vs authorization

Study authentication vs authorization through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

2. Password hashing

Study password hashing through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

3. Sessions

Study sessions through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

4. JWT

Study jwt through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

5. Refresh tokens

Study refresh tokens through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

6. CORS

Study cors through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

Worked example

Authorization middleware concept

```
function requireAuth(req, res, next) {
  const token = readTokenFromRequest(req);

  if (!token) {
    return res.status(401).json({ error: "Authentication required" });
  }

  // Verify token/session here.
  req.user = decodedUser;
  next();
}
```

Practice variation: change one input, introduce one failure, predict the result, then run it. Rewrite the example from memory afterward.

Comparison table

Concept	Meaning	Practical use
Authentication	Who are you?	Identity
Authorization	What can you do?	Permissions
Hashing	One-way password protection	Store password hashes, not plaintext

Common mistakes

- Plaintext passwords
- Confusing authentication with authorization
- Long-lived tokens with no strategy
- Trusting client-side validation
- Misconfigured CORS

Best practices

- Keep responsibilities clear and small.
- Validate inputs at system boundaries.
- Prefer readable code over clever code.
- Test important success and failure paths.
- Use official documentation for current API/platform behavior.
- Keep secrets and credentials out of source control.
- Document important trade-offs so another developer can reproduce your reasoning.

Extended practical lab

Complete these tasks in order. The goal is to turn passive knowledge into evidence that you can actually use the topic.

Lab 1

- Create a threat model for a simple login flow.
- Write down what you expected, what happened, and why.

Lab 2

- Decide how passwords are hashed.
- Write down what you expected, what happened, and why.

Lab 3

- Choose an authentication mechanism and document why.
- Write down what you expected, what happened, and why.

Lab 4

- List what happens when a token expires.
- Write down what you expected, what happened, and why.

Lab 5

- Test an unauthorized request.
- Write down what you expected, what happened, and why.

Lab 6

- Test malformed input and verify the server does not trust the client.
- Write down what you expected, what happened, and why.

Mini-project

Add registration/login to a small API using password hashing and short-lived access tokens. Document the security decisions and explicitly avoid storing plaintext passwords.

Acceptance criteria

- A new developer can understand the project from its README.
- The main happy path works from start to finish.
- At least three edge/failure cases are tested.
- The project has meaningful Git commits.
- The project includes a short explanation of design choices.
- If deployment applies, production behavior is verified rather than assumed.

Interview preparation

Practice answering these aloud. A strong answer should define the concept, give a concrete example, mention one trade-off, and explain when you would use it.

Authentication vs authorization?

- Answer structure: definition → example → trade-off/limitation → practical use.

Why hash passwords?

- Answer structure: definition → example → trade-off/limitation → practical use.

What is a JWT?

- Answer structure: definition → example → trade-off/limitation → practical use.

What happens when a token expires?

- Answer structure: definition → example → trade-off/limitation → practical use.

What is CORS?

- Answer structure: definition → example → trade-off/limitation → practical use.

Why is client-side validation insufficient?

- Answer structure: definition → example → trade-off/limitation → practical use.

Debugging checklist

- Reproduce the problem consistently.
- Reduce it to the smallest failing example.
- Inspect inputs at each boundary.
- Check the first incorrect value, not only the final error.
- Read the complete error message and stack trace.
- Change one thing at a time.
- Retest the original failure after the fix.
- Add a regression test when the bug is important.

Glossary

Term	Your own definition	Example/use
Authentication	Write this in your own words.	Give one project example.
Authorization	Write this in your own words.	Give one project example.
Hash	Write this in your own words.	Give one project example.
Salt	Write this in your own words.	Give one project example.
JWT	Write this in your own words.	Give one project example.
Session	Write this in your own words.	Give one project example.
Refresh token	Write this in your own words.	Give one project example.
CORS	Write this in your own words.	Give one project example.

Term	Your own definition	Example/use
CSRF	Write this in your own words.	Give one project example.
XSS	Write this in your own words.	Give one project example.

Self-test

- Explain Authentication vs authorization without using its textbook definition.
- Show a small example of Password hashing.
- What is the most important boundary in this topic?
- What is one failure mode and how would you diagnose it?
- What trade-off should a developer know before using this technique?
- How would you test the normal path and an edge case?
- How does this topic connect to the next roadmap step?
- What would you change if the system had 100× more users/data?
- What part of this topic would you explain differently to a beginner?
- What can you now build that you could not build before studying this step?

Final checklist

- I can explain and demonstrate Authentication vs authorization.
- I can explain and demonstrate Password hashing.
- I can explain and demonstrate Sessions.
- I can explain and demonstrate JWT.
- I can explain and demonstrate Refresh tokens.
- I can explain and demonstrate CORS.
- I can explain and demonstrate CSRF/XSS.
- I can explain and demonstrate Input validation.
- I can explain and demonstrate Secrets.
- I can explain and demonstrate Rate limiting.
- I completed the practical lab.
- I built or extended the mini-project.
- I tested at least three failure/edge cases.
- I can explain one trade-off.
- I can answer the interview questions without notes.
- I know where to find the authoritative documentation.

Recommended documentation

- https://developer.mozilla.org/en-US/docs/Web/Security
- https://owasp.org/

Recommended videos

- Dave Gray — JWT Authentication | Node JS and Express tutorials for Beginners
  https://www.youtube.com/watch?v=favjC6EKFgw (NEW)

Source note: resource references are based on the existing CareerCompass database plus verified web research. For changing APIs, versions, deployment settings and security guidance, consult the linked official documentation before production use.